Amundsen.io

Configuración del servicio de metadatos

C. Enrique Israel Torres Mendoza

Dirección del Laboratorio de Ciencia de Datos y Métodos Modernos de Producción de Información

2023

Introducción.

En esta presentación se pretende trabajar con el servicio de metadatos de amundsen, revisando la guía de instalación y documentación para hacer las configuraciones necesarias dentro de los archivos de la aplicación.
Si bien ya se ah podido levantar el servicio de frontend mediante el contenedor con podman y también el servicio search con el uso de elastic search.
Para poder trabajar con la generación del catalogo de datos es necesario que el servicio de metadatos de amundsen consuma estos metadatos desde algún backend, se pretende que este backend sea con Neo4j

Quick start

Para empezar con amundsen y manejar su interfaz con datos ficticios es necesario que tengamos el repositorio de amundsen. Se descarga con el siguiente comando:

```
# For Neo4j Backend
$ docker-compose -f docker-amundsen.yml up

# For Atlas
$ docker-compose -f docker-amundsen-atlas.yml up
```

Para nuestro caso se ingresa el comando para backend de Neo4j:

Este comando se encarga de descargar las imágenes de amundsen necesarias para empezar con la prueba rápida, descarga la imagen de amundsen de frontend, search y metadata, también descarga una imagen de librería de Neo4j y también una imagen de elasticsearch.

Quick start

```
[root@localhost metadata_service]# podman ps
CONTAINER ID  IMAGE                                      COMMAND               CREATED      STATUS             PORTS                                                  NAMES
b84531289e14  docker.io/library/neo4j:3.5.26             neo4j                 3 hours ago  Up 21 seconds ago  0.0.0.0:7474->7474/tcp, 0.0.0.0:7687->7687/tcp         neo4j_amundsen
1f7ac5ab448a  docker.io/library/elasticsearch:8.0.0      eswrapper             3 hours ago  Up 29 seconds ago  0.0.0.0:9200->9200/tcp                                 es_amundsen
b445641f6228  docker.io/amundsendev/amundsen-search:4.0.2   gunicorn -w 2 --b...  3 hours ago  Up 25 seconds ago  0.0.0.0:5001->5000/tcp                                 amundsensearch
3d4028d3a0ea  docker.io/amundsendev/amundsen-metadata:3.11.0  gunicorn -w 2 --b...  3 hours ago  Up 27 seconds ago  0.0.0.0:5002->5000/tcp                                 amundsenmetadata
c217ff8a6737  docker.io/amundsendev/amundsen-frontend:4.2.0   gunicorn -w 2 --b...  3 hours ago  Up 16 seconds ago  0.0.0.0:5000->5000/tcp                                 amundsenfrontend
[root@localhost metadata_service]#
```

Dentro de la carpeta de amundsen tenemos una carpeta que almacena todo el servicio de metadatos, dentro de esta carpeta también se encuentra una carpeta llamada “metadata_service” el cual contiene la configuración que indica de donde se van a extraer los metadatos. El archivo donde se encuentra esta configuración es config.py

```
[enriquetorres@localhost metadata]$ cd metadata_service/
[enriquetorres@localhost metadata_service]$ ls -l
total 44
drwxrwxr-x. 3 enriquetorres enriquetorres 4096 Mar 27 13:06 api
drwxrwxr-x. 2 enriquetorres enriquetorres   47 Mar 27 13:06 cli
drwxrwxr-x. 2 enriquetorres enriquetorres   46 Mar 27 13:06 client
-rw-rw-r--. 1 enriquetorres enriquetorres 6461 Mar 27 13:30 config.py
-rw-rw-r--. 1 enriquetorres enriquetorres  813 Mar 27 13:06 deprecations.py
drwxrwxr-x. 2 enriquetorres enriquetorres  189 Mar 27 13:06 entity
-rw-rw-r--. 1 enriquetorres enriquetorres  206 Mar 27 13:06 exception.py
-rw-rw-r--. 1 enriquetorres enriquetorres 9310 Mar 27 13:06 __init__.py
-rw-rw-r--. 1 enriquetorres enriquetorres  391 Mar 27 13:06 metadata_wsgi.py
drwxrwxr-x. 2 enriquetorres enriquetorres 4096 Mar 27 13:06 proxy
-rw-rw-r--. 1 enriquetorres enriquetorres  194 Mar 27 13:06 util.py
[enriquetorres@localhost metadata_service]$
```

Editando archivo config.py

Para poder editar este archivo y hacer las configuraciones necesarias entramos con el comando “vi config.py”

```
[root@localhost metadata_service]# vi config.py
[root@localhost metadata_service]#
```

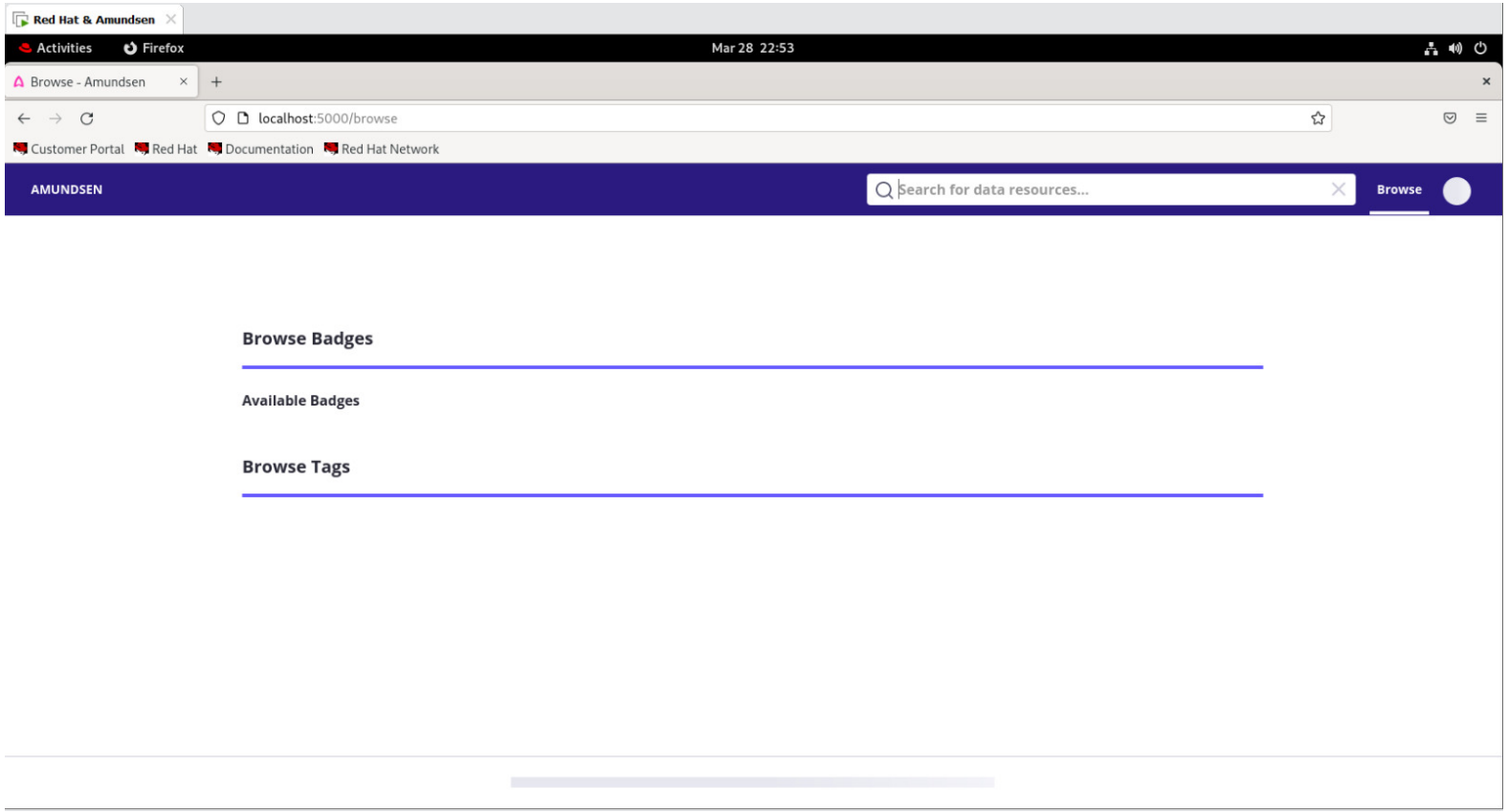
Inmediatamente se abrirá el editor que usa Linux y podemos hacer modificaciones al archivo, se debe configurar el proxy_client, proxy_host, proxy_port, proxy_user y proxi_password. La configuración queda de la siguiente manera:

Editando archivo config.py

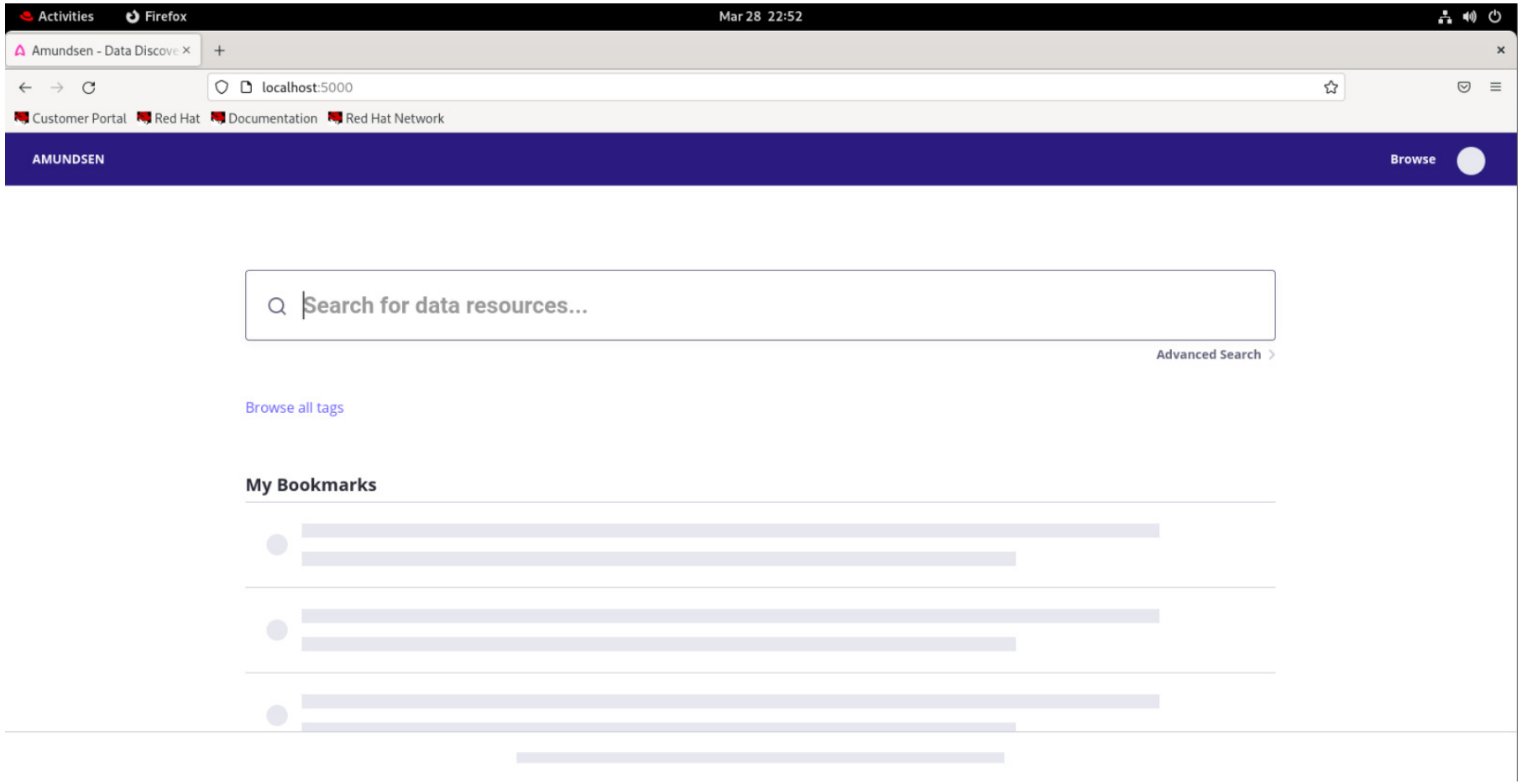
```
# Copyright Contributors to the Amundsen project.
# SPDX-License-Identifier: Apache-2.0

import distutils.util
import logging
import os
from typing import Any, Callable, Dict, List, Optional, Set  # noqa: F401

import boto3
from flask import Flask  # noqa: F401

from metadata_service.entity.badge import Badge


# PROXY configuration keys
PROXY_HOST = f'bolt://{LOCAL_HOST}'
PROXY_PORT = 7687
PROXY_USER = 'neo4juser'
PROXY_PASSWORD = 'password'
PROXY_ENCRYPTED = 'PROXY_ENCRYPTED'
PROXY_VALIDATE_SSL = 'PROXY_VALIDATE_SSL'
PROXY_DATABASE_NAME = 'PROXY_DATABASE_NAME'
PROXY_CLIENT = PROXY_CLIENTS['NEO4J']
PROXY_CLIENT_KWARGS = 'PROXY_CLIENT_KWARGS'

PROXY_CLIENTS = {
    'NEO4J': 'metadata_service.proxy.neo4j_proxy.Neo4jProxy',
    'ATLAS': 'metadata_service.proxy.atlas_proxy.AtlasProxy',
    'NEPTUNE': 'metadata_service.proxy.neptune_proxy.NeptuneGremlinProxy',
    'MYSQL': 'metadata_service.proxy.mysql_proxy.MySQLProxy'
}
```

Visualizando amundsen

Amundsen es capaz de correr su aplicación pero no muestra datasets ya que actualmente no se ha instalado Neo4j, con lo cual amundsen no tiene datos existentes del backend



Visualizando amundsen



Conclusión.

Hasta ahora se ah logrado implementar el servicio de frontend de amundsen con podman y también el servicio de búsqueda, se pretende trabajar con neo4j para poder integrar los datos que se tienen en neo4j a amundsen y tener una visualización de estos desde amundsen y aprovechar los controles que tiene amundsen para los diferentes datasets, tableros y tablas con los que cuenta amundsen y empezar con pruebas para la generación de un catalogo de datos.

GRACIAS

Conociendo
México

800 111 46 34
www.inegi.org.mx
atencion.usuarios@inegi.org.mx

INEGI Informa